Dimensionality reduction and starting with scikit-learn

Dimensionality reduction

Transformation of data from a high-dimensional space into a low-dimensional space.

Why?

- Avoid curse of dimensionality for sparse data.
- Might be intractable to compute otherwise.
- To be able to plot in 1D, 2D or 3D (the examples in this lecture).

(For the plotting: transform to very low-dimensional space)

scikit-learn example

```
1  import numpy as np
2  from sklearn.decomposition import TruncatedSVD
3
4  from text_search import WordVectors
5
6  if __name__ == '__main__':
7
8      wv = WordVectors("en.vec.gz") # My own class
9      all_words = list(wv.vectors.keys())
10     vector_space = \
11         np.array([wv.get_vector(word) for word in all_words])
12     print("vector_space shape:", vector_space.shape)
13     print(all_words[2000], vector_space[2000])
14
15     print("----")
16     svd = TruncatedSVD(n_components=2, random_state=1)
17     two_dims = svd.fit_transform(vector_space)
18     print("two_dims shape:", two_dims.shape)
19     print(all_words[2000], two_dims[2000])
```

scikit-learn example

```
wv = WordVectors("en.vec.gz") # My own class
all_words = list(wv.vectors.keys())
vector_space = \
        np.array([wv.get_vector(word) for word in all_words])
print("vector_space shape:", vector_space.shape)
print(all_words[2000], vector_space[2000])

print("----")
svd = TruncatedSVD(n_components=2, random_state=1)
two_dims = svd.fit_transform(vector_space)
print("two_dims shape:", two_dims.shape)
print(all_words[2000], two_dims[2000])
```

```
vector_space shape: (362146, 300)
accessibility [-0.0631 -0.0455 ... 0.039   0.0399]
----
two_dims shape: (362146, 2)
accessibility [-0.01007749  0.01432768]
```

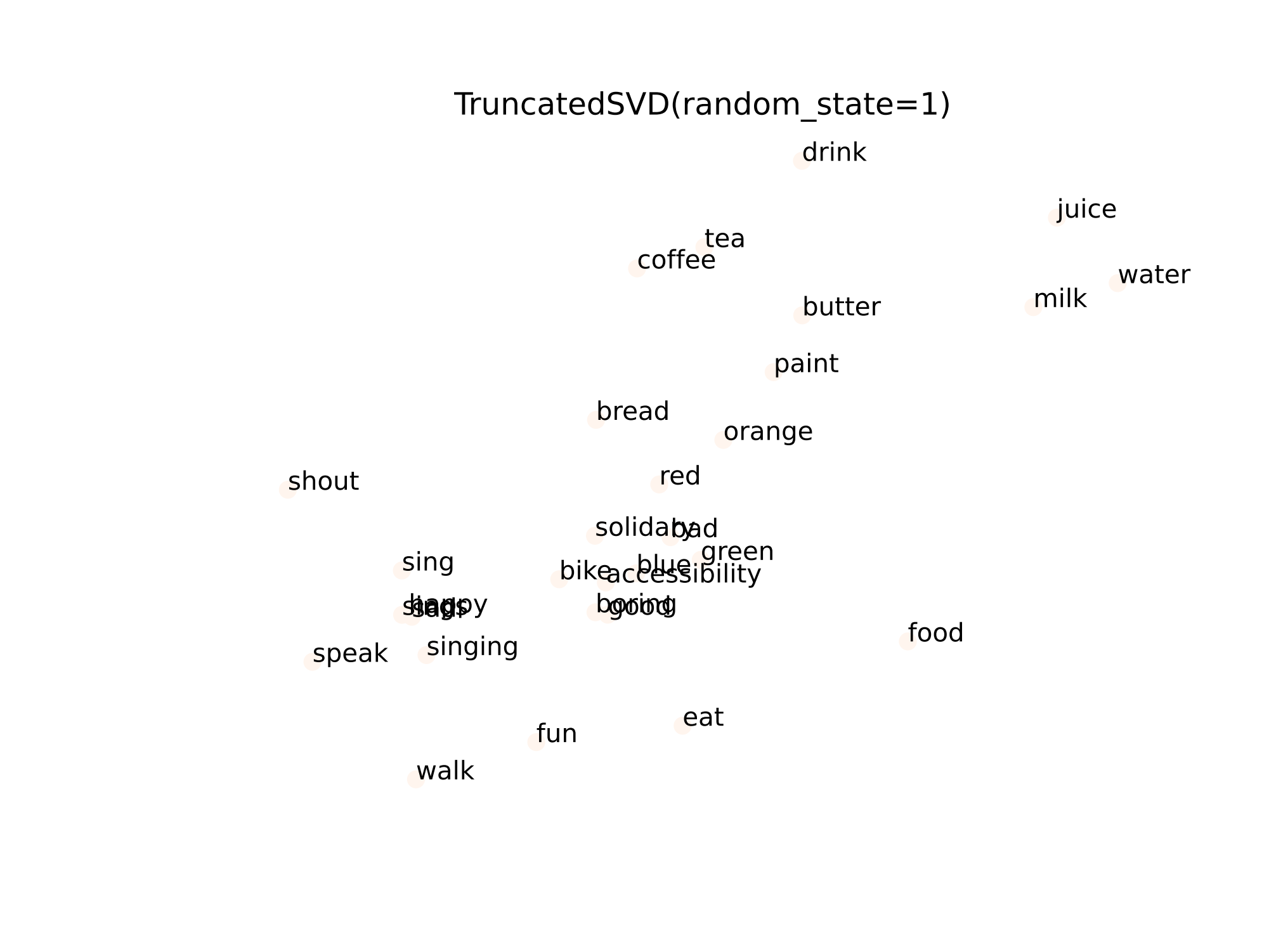
Plot some of the words

```
import matplotlib.pyplot as plt

to_plot = ["coffee", "tea", "juice", "bread", "sing",\
          "red", "green", "blue", "walk", "shout", "speak",\
         "orange", "happy", "sad", "good", "bad",\
         "eat", "water", "butter", "milk", "accessibility", \
         "fun", "boring", "singing", "sings",\
         "food", "solidary", "drink", "bike", "paint"]

for word in to_plot:
    word_index = all_words.index(word)
    (x, y) = two_dims[word_index]
    plt.scatter(x, y, color="seashell")
    plt.annotate(word, (x,y))

plt.axis('off')
plt.title(svd)
plt.savefig("words_1.pdf", format="pdf")
```


TruncatedSVD(random_state=1)
drink
juice
tea
coffee
water
milk
butter
paint
bread
orange
shout
red
solidarity
bad
green
sing
blue
bike
accessibility
happy
boring
good
food
speak
singing
eat
fun
walk

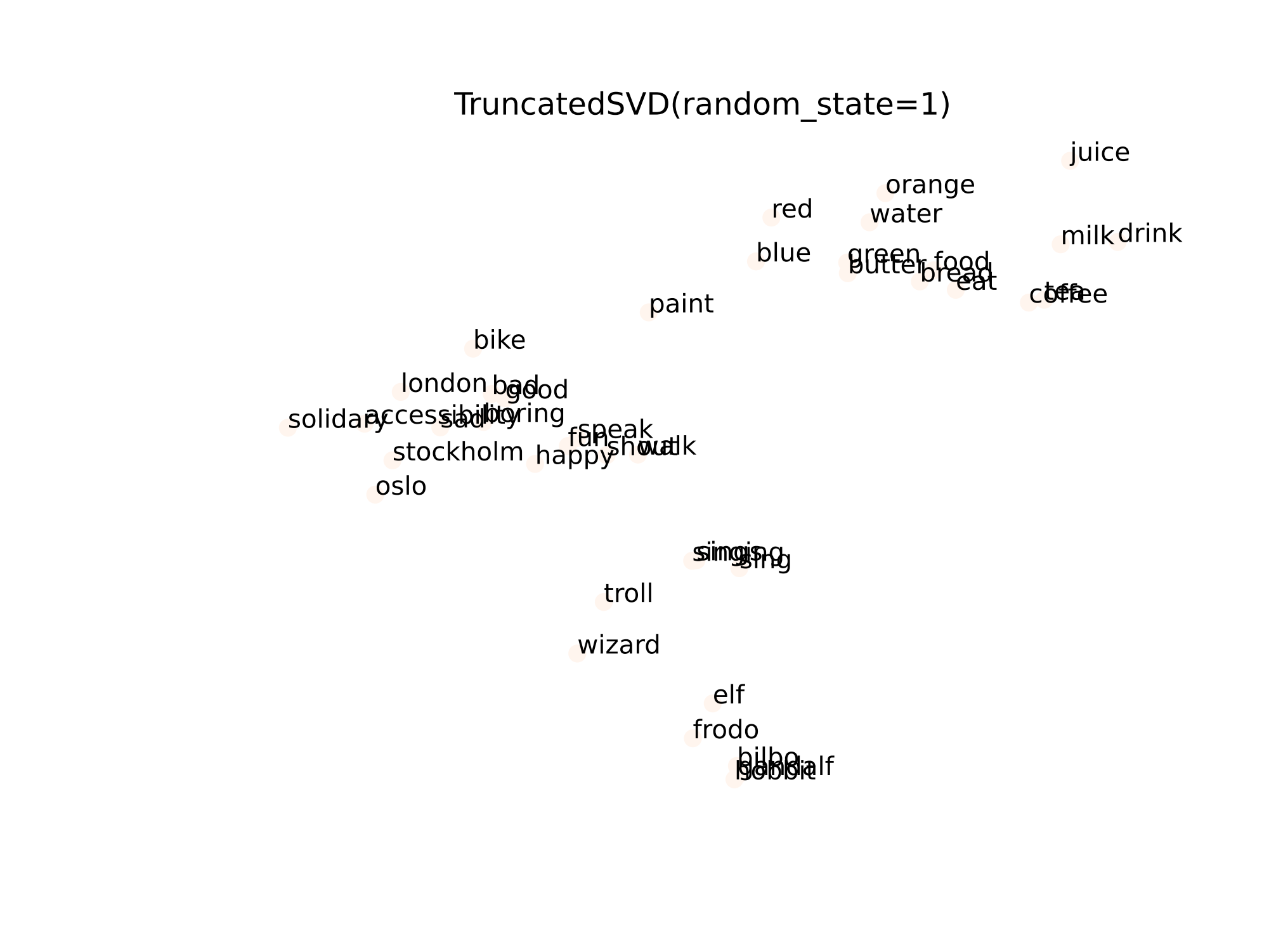
Make function for reducing dimensionality and plot

Also: Do dimensionality reduction on a space constructed only from the words included in the visualisation (no right or wrong for the visualisation, depends on what you want to show).

```
1  def plot_words(words, w_vect, dim_reductor, file_name):
2      plt.clf()
3      vector_space = \
4          np.array([w_vect.get_vector(w) for w in words])
5      two_dims = dim_reductor.fit_transform(vector_space)
6
7      for word, (x, y) in zip(words, two_dims):
8          plt.scatter(x, y, color="seashell")
9          plt.annotate(word, (x,y))
10
11     plt.axis('off')
12     plt.title(dim_reductor)
13     plt.savefig(file_name, format="pdf")
```

Call function

```
1  import numpy as np
2  from sklearn.decomposition import TruncatedSVD
3  import matplotlib.pyplot as plt
4  from text_search import WordVectors
5
6  if __name__ == '__main__':
7      wv = WordVectors("en.vec.gz") # My own class
8      to_plot = ["coffee", "tea", "juice", "bread", "sing",\
9       "red", "green", "blue", "walk", "shout", "speak",\
10     "orange", "happy", "sad", "good", "bad",\
11     "eat", "water", "butter", "milk", "accessibility", \
12     "fun", "boring", "singing", "sings",\
13     "food", "solidary", "drink", "bike", "paint",\
14     "bilbo", "frodo", "wizard", "troll", "hobbit",\
15     "gandalf", "elf", "stockholm", "london", "oslo" ]
16
17     svd = TruncatedSVD(n_components=2, random_state=1)
18     plot_words(to_plot, wv, svd, "small_space.pdf")
```

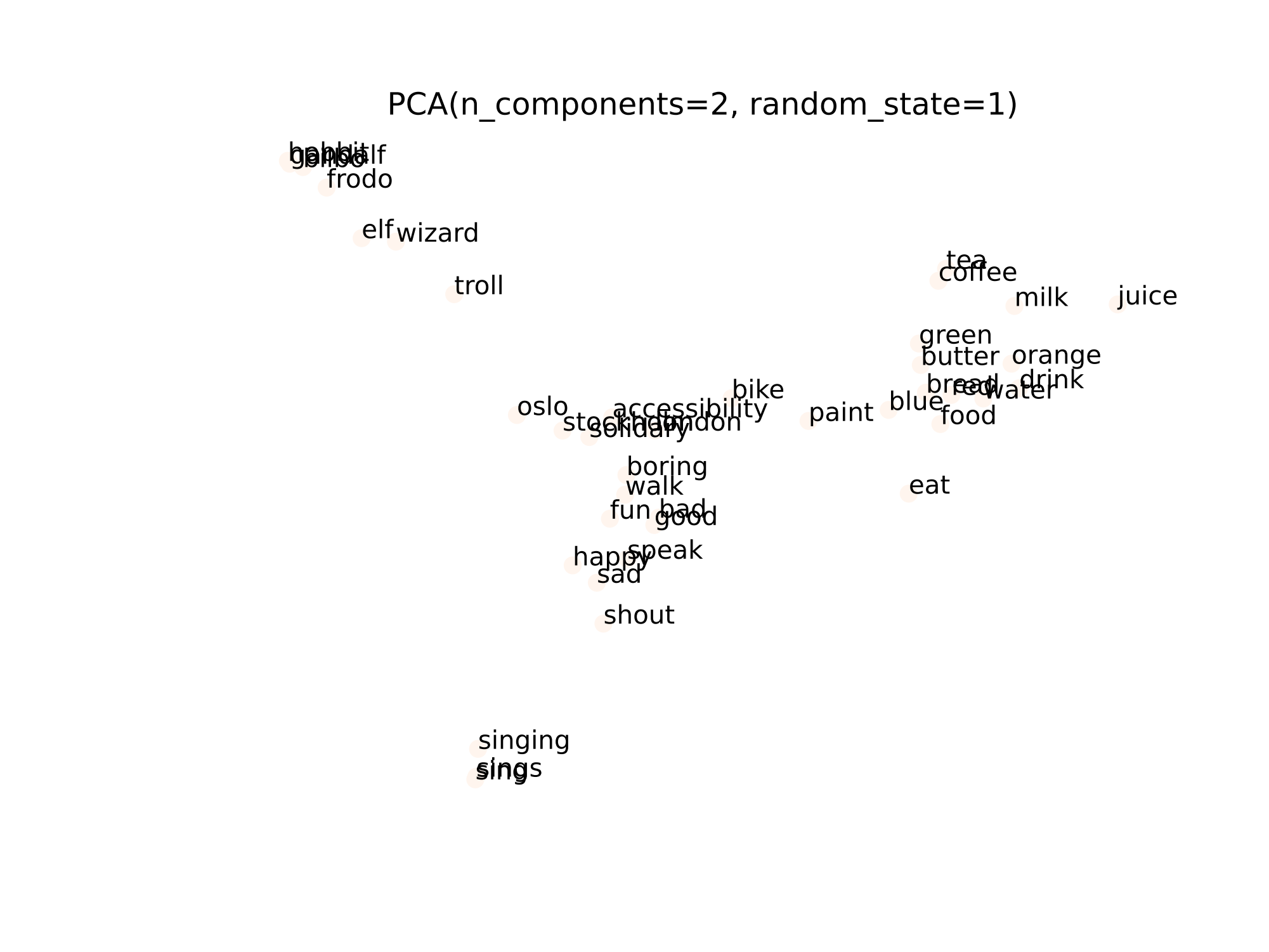

TruncatedSVD(random_state=1)
juice
orange
red
water
milk
drink
blue
green
butter
food
bread
eat
coffee
tea
paint
bike
london
bad
good
solidary
accessibility
sad
boring
speak
fun
show
walk
stockholm
happy
oslo
sing
troll
wizard
elf
frodo
bilbo
gandalf
hobbit

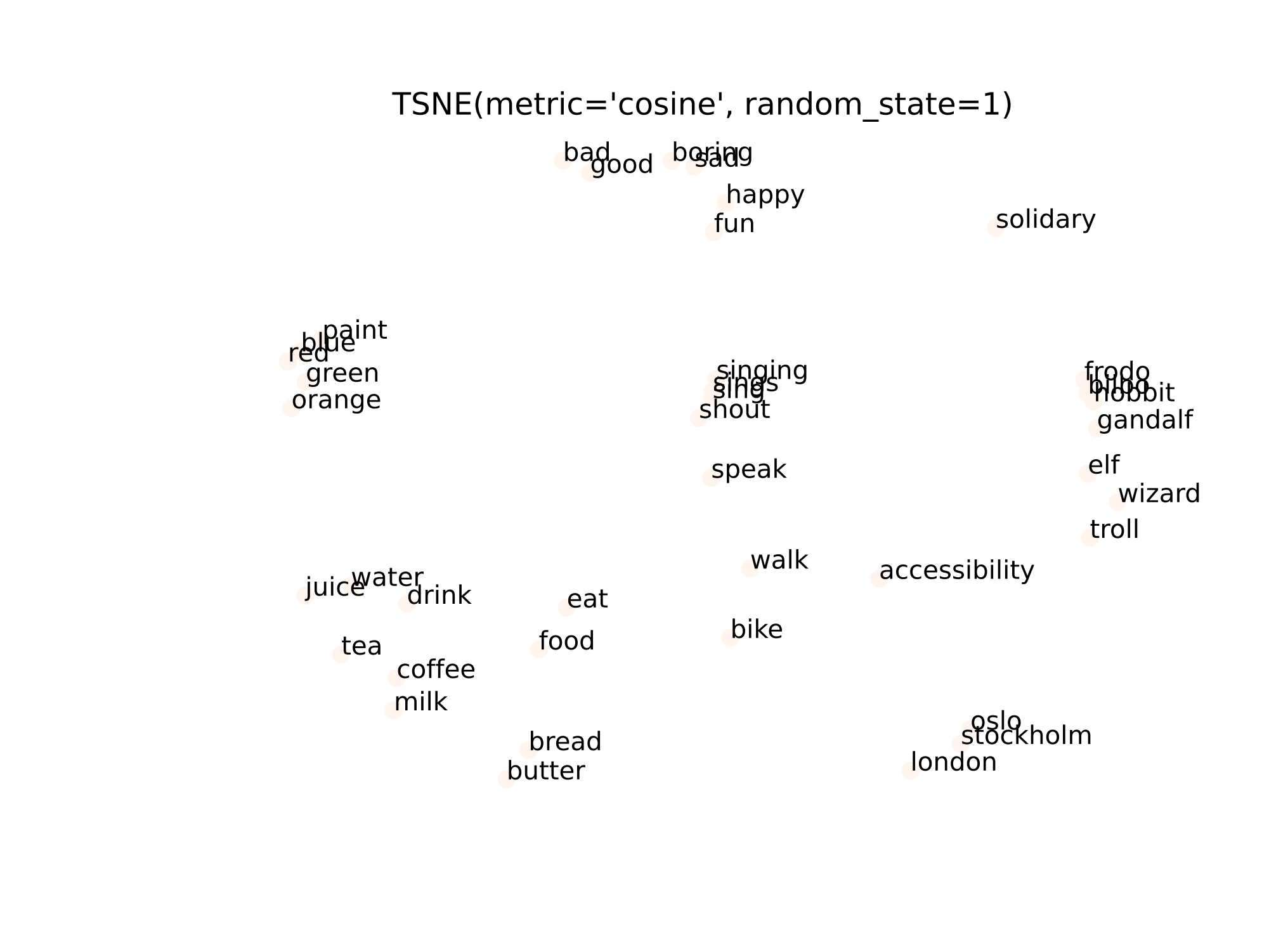
Try other dimensionality reduction techniques

```
if __name__ == '__main__':
    wv = WordVectors("en.vec.gz") # My own class
    to_plot = ["coffee", "tea", "juice", "bread", "sing",\
     "red", "green", "blue", "walk", "shout", "speak",\
    "orange", "happy", "sad", "good", "bad",\
    "eat", "water", "butter", "milk", "accessibility", \
    "fun", "boring", "singing", "sings",\
    "food", "solidary", "drink", "bike", "paint",\
    "bilbo", "frodo", "wizard", "troll", "hobbit",\
    "gandalf", "elf", "stockholm", "london", "oslo" ]

    dim_reductors = [
        ("svd.pdf",\
            TruncatedSVD(n_components=2, random_state=1)),
        ("pca.pdf",\
            PCA(n_components=2, random_state=1)),
        ("tsne.pdf",\
            TSNE(n_components=2, random_state=1,\
             metric = "cosine"))]

    for file_name, dim_reductor in dim_reductors:
        plot_words(to_plot, wv, dim_reductor, file_name)
```


PCA(n_components=2, random_state=1)
frodo
elf
wizard
troll
tea
coffee
milk
juice
green
butter
orange
drink
bike
oslo
accessibility
paint
blue
food
boring
walk
eat
fun
bad
good
speak
happy
sad
shout
singing
sings
sing


TSNE(metric='cosine', random_state=1)
bad
good
boring
sad
happy
fun
solidary
paint
blue
red
green
orange
singing
sings
sing
shout
frodo
bilbo
hobbit
gandalf
speak
elf
wizard
troll
walk
accessibility
juice
water
drink
eat
bike
tea
food
coffee
milk
oslo
stockholm
bread
butter
london

Classes used

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.decomposition import TruncatedSVD
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE

from text_search import WordVectors
```

TruncatedSVD

class **sklearn.decomposition.TruncatedSVD(*n_components*=*2*, ***, *algorithm*='*randomized*', *n_iter*=*5*, *n_oversamples*=*10*, *power_iteration_normalizer*='*auto*', *random_state*=*None*, *tol*=*0.0*)** [source]

Dimensionality reduction using truncated SVD (aka LSA).

This transformer performs linear dimensionality reduction by means of truncated singular value decomposition (SVD). Contrary to PCA, this estimator does not center the data before computing the singular value decomposition. This means it can work with sparse matrices efficiently.

In particular, truncated SVD works on term count/tf-idf matrices as returned by the vectorizers in `sklearn.feature_extraction.text`. In that context, it is known as latent semantic analysis (LSA).

This estimator supports two algorithms: a fast randomized SVD solver, and a "naive" algorithm that uses ARPACK as an eigensolver on `X * X.T` or `X.T * X`, whichever is more efficient.

Read more in the User Guide.

Parameters:

n_components : *int, default=2*

Desired dimensionality of output data. If algorithm='arpack', must be strictly less than the number of features. If algorithm='randomized', must be less than or equal to the number of features. The default value is useful for visualisation. For LSA, a value of 100 is recommended.

https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.TruncatedSVD.html

Source code

```
27
28  class TruncatedSVD(ClassNamePrefixFeaturesOutMixin, TransformerMixin, BaseEstimator):
29      """Dimensionality reduction using truncated SVD (aka LSA).
30
31      This transformer performs linear dimensionality reduction by means of
32      truncated singular value decomposition (SVD). Contrary to PCA, this
33      estimator does not center the data before computing the singular value
34      decomposition. This means it can work with sparse matrices
35      efficiently.
36
37      In particular, truncated SVD works on term count/tf-idf matrices as
38      returned by the vectorizers in :mod:`sklearn.feature_extraction.text`. In
39      that context, it is known as latent semantic analysis (LSA).
40
41      This estimator supports two algorithms: a fast randomized SVD solver, and
42      a "naive" algorithm that uses ARPACK as an eigensolver on `X * X.T` or
43      `X.T * X`, whichever is more efficient.
44
45      Read more in the :ref:`User Guide <LSA>`.
46
```

https://github.com/scikit-learn/scikit-learn/blob/d3898d9d5/sklearn/decomposition/_truncated_svd.py#L28

Source code

Note how _ is used to signal what is **not** part of the public interface of scikit-learn.

PCA class:

```
class PCA(_BasePCA):
    """Principal component analysis (PCA).

    Linear dimensionality reduction using Singular Value Decomposition of the
    data to project it to a lower dimensional space. The input data is centered
    but not scaled for each feature before applying the SVD.
```

https://github.com/scikit-learn/scikit-learn/blob/d3898d9d5/sklearn/decomposition/_pca.py#L113

_BasePCA class

```
class _BasePCA(
    ClassNamePrefixFeaturesOutMixin, TransformerMixin, BaseEstimator, metaclass=ABCMeta
):
    """Base class for PCA methods.

    Warning: This class should not be used directly.
    Use derived classes instead.
    """
```

https://github.com/scikit-learn/scikit-learn/blob/main/sklearn/decomposition/_base.py

Decomposition

In `__init__.py` are the classes in decomposition made available.

scikit-learn / sklearn / decomposition / __init__.py

lesteve MNT Switch to absolute imports enforced by ruff (#31847) ✓

Code | Blame | 50 lines (46 loc) · 1.49 KB | Raw

```python
"""Matrix decomposition algorithms.

These include PCA, NMF, ICA, and more. Most of the algorithms of this module can be
regarded as dimensionality reduction techniques.
"""

# Authors: The scikit-learn developers
# SPDX-License-Identifier: BSD-3-Clause

from sklearn.decomposition._dict_learning import (
    DictionaryLearning,
    MiniBatchDictionaryLearning,
    SparseCoder,
    dict_learning,
    dict_learning_online,
    sparse_encode,
)
from sklearn.decomposition._factor_analysis import FactorAnalysis
from sklearn.decomposition._fastica import FastICA, fastica
from sklearn.decomposition._incremental_pca import IncrementalPCA
from sklearn.decomposition._kernel_pca import KernelPCA
from sklearn.decomposition._lda import LatentDirichletAllocation
from sklearn.decomposition._nmf import NMF, MiniBatchNMF, non_negative_factorization
from sklearn.decomposition._pca import PCA
from sklearn.decomposition._sparse_pca import MiniBatchSparsePCA, SparsePCA
from sklearn.decomposition._truncated_svd import TruncatedSVD
from sklearn.utils.extmath import randomized_svd
```

https://github.com/scikit-learn/scikit-learn/tree/d3898d9d57aeb1e960d266613a2e31b07bca39d7/sklearn/decomposition

Source code for TSNE

```
class TSNE(ClassNamePrefixFeaturesOutMixin, TransformerMixin, BaseEstimator):
    """T-distributed Stochastic Neighbor Embedding.

    t-SNE [1] is a tool to visualize high-dimensional data. It converts
    similarities between data points to joint probabilities and tries
    to minimize the Kullback-Leibler divergence between the joint
    probabilities of the low-dimensional embedding and the
    high-dimensional data. t-SNE has a cost function that is not convex,
    i.e. with different initializations we can get different results.

    It is highly recommended to use another dimensionality reduction
    method (e.g. PCA for dense data or TruncatedSVD for sparse data)
    to reduce the number of dimensions to a reasonable amount (e.g. 50)
    if the number of features is very high. This will suppress some
    noise and speed up the computation of pairwise distances between
    samples. For more tips see Laurens van der Maaten's FAQ [2].

    Read more in the :ref:`User Guide <t_sne>`.
```

https://github.com/scikit-learn/scikit-learn/blob/d3898d9d5/sklearn/manifold/_t_sne.py#L560

One of the superclasses: TransformerMixin

```
class TransformerMixin(_SetOutputMixin):
    def fit_transform(self, X, y=None, **fit_params):
        """
        Fit to data, then transform it.

        Fits transformer to `X` and `y`
        with optional parameters `fit_params`
        and returns a transformed version of `X`.

        Parameters
        ----------
        X : array-like of shape (n_samples, n_features)
            Input samples.

        y :  array-like of shape (n_samples,)
            or (n_samples, n_outputs), \
                default=None
            Target values
            (None for unsupervised transformations).
```

https://github.com/scikit-learn/scikit-learn/blob/d3898d9d57aeb1e960d266613a2e31b07bca39d7/sklearn/base.py

We call `fit_transform` for three different subclasses of `TransformerMixin`, and get different transformations. (Our `plot_words` method doesn't have to care what happens when `fit_transform` is called, it just uses the result, whatever it is)

```
def plot_words(words, w_vect, dim_reductor, file_name):
...
    vector_space = \
        np.array([w_vect.get_vector(w) for w in words])
    two_dims = dim_reductor.fit_transform(vector_space)
...

if __name__ == '__main__':
    dim_reductors = [
        ("svd.pdf",\
            TruncatedSVD(n_components=2, random_state=1)),
        ("pca.pdf",\
            PCA(n_components=2, random_state=1)),
        ("tsne.pdf",\
            TSNE(n_components=2, random_state=1,\
             metric = "cosine"))]

    for file_name, dim_reductor in dim_reductors:
        plot_words(to_plot, wv, dim_reductor, file_name)
```

But actually, Python doesn't care

As long as the object we use as argument to `plot_words` has a `fit_transform` method, it will work. Python doesn't care if the object is an instance of `TransformerMixin` or not.

We can write our own class that has a `fit_transform` method, and that is **not** a subclass of `TransformerMixin`.

```
class SimpleReducer():

    def __init__(self, n_components):
        self.n_components = n_components

    def fit_transform(self, X):
        return X[:,:self.n_components]

    def __str__(self):
        return\
         f"SimpleReducer(n_components={self.n_components})"
```

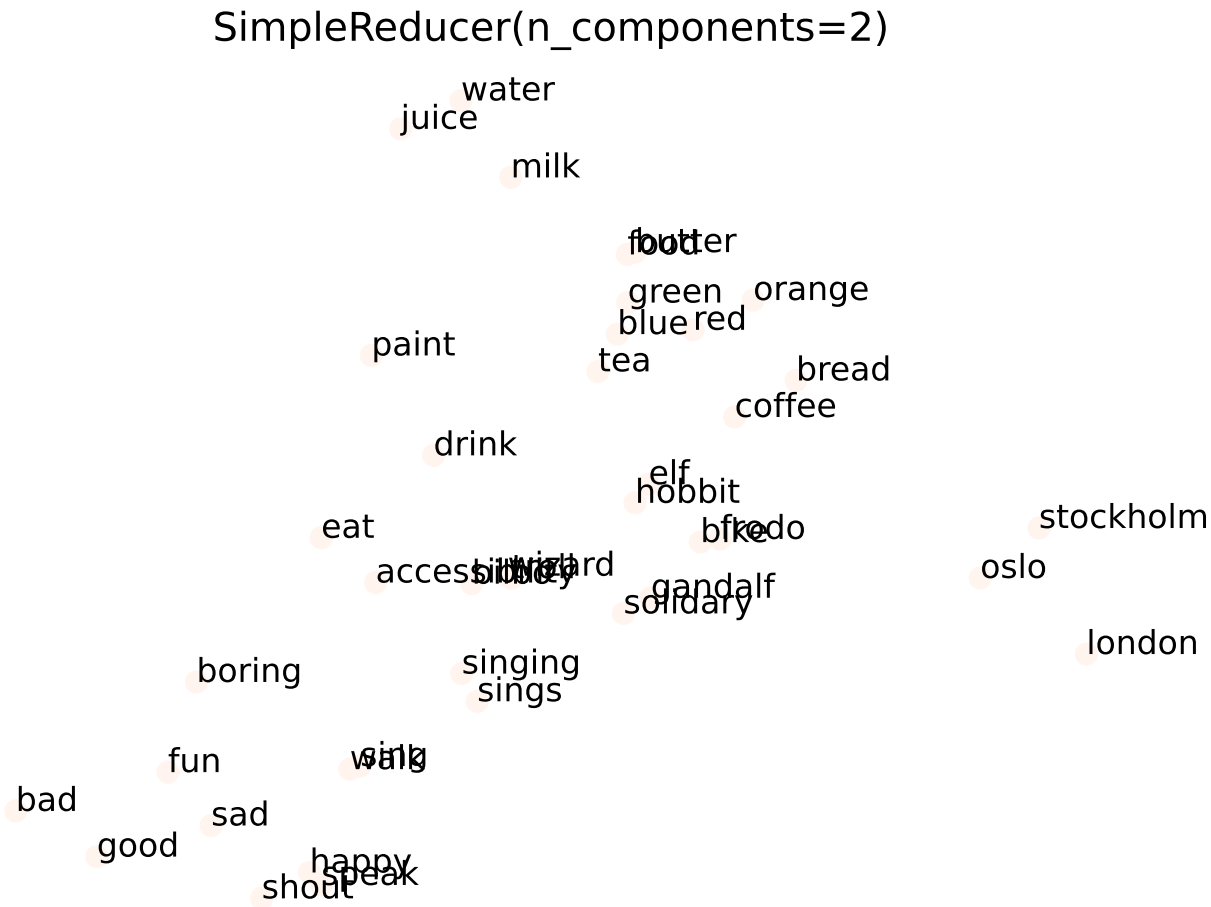
Our `SimpleReducer` works just as fine to send to `plot_words`

```
if __name__ == '__main__':
    wv = WordVectors("en.vec.gz") # My own class
    to_plot = ["coffee", "tea", "juice", "bread",
    "gandalf", "elf", "stockholm", "london", "oslo" ]

    dim_reductors = [
        ("svd.pdf",\
            TruncatedSVD(n_components=2, random_state=1)),
        ("pca.pdf",\
            PCA(n_components=2, random_state=1)),
        ("tsne.pdf",\
            TSNE(n_components=2, random_state=1,\
             metric = "cosine")),
        ("simple.pdf",\
            SimpleReducer(n_components=2))]

    for file_name, dim_reductor in dim_reductors:
        plot_words(to_plot, wv, dim_reductor, file_name)
```

The vectors we use have been constructed using dimensionality reduction from vectors with a larger dimension, and where the method used orders the elements according to importance.



(https://aclanthology.org/S17-2008/)

Try the two last elements instead

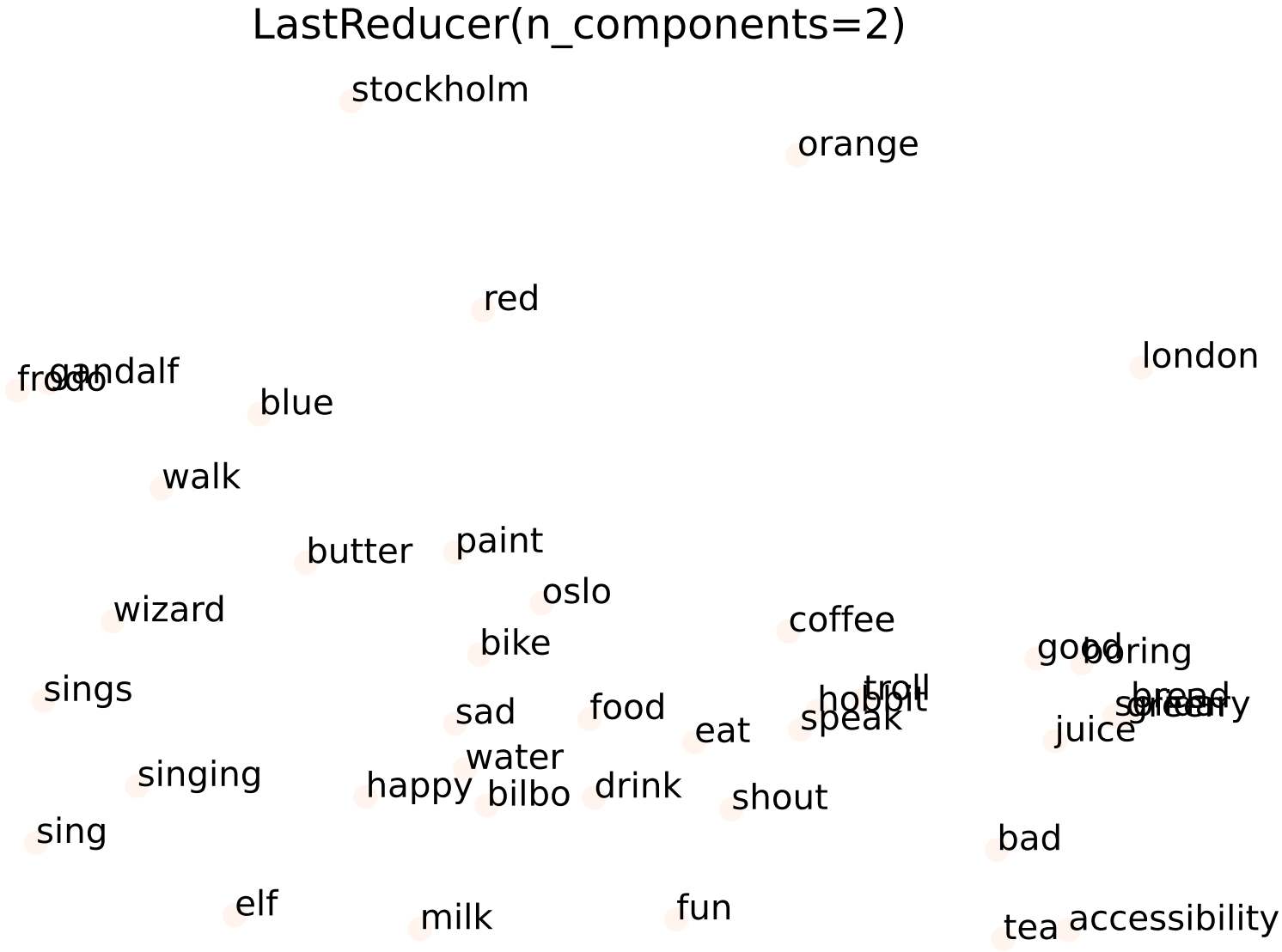
```
class LastReducer(SimpleReducer):
    def fit_transform(self, X):
        return X[:,-self.n_components:]

    def __str__(self):
        return\
         f"LastReducer(n_components={self.n_components})"
```

Try the two last elements instead

```
if __name__ == '__main__':
    dim_reductors = [
          ("svd.pdf",\
              TruncatedSVD(n_components=2, random_state=1)),
          ("pca.pdf",\
              PCA(n_components=2, random_state=1)),
          ("tsne.pdf",\
              TSNE(n_components=2, random_state=1,\
               metric = "cosine")),
          ("simple.pdf",\
              SimpleReducer(n_components=2)),
          ("last.pdf",\
              LastReducer(n_components=2))]
     for file_name, dim_reductor in dim_reductors:
          plot_words(to_plot, wv, dim_reductor, file_name)
```

Not as intelligible



Sparse arrays

For the lab (distinction part), you will construct vectors with very, very many columns (one for each sentence in the data).
And then use dimensionality reduction to reduce the vectors to 300 columns.

```
      s_0 s_1 s2 s3 s4 s5 s6 s7 s8

a:   [2, 0, 0, 0, 1, 1, 1, 0, 0 ...]
and: [0, 4, 0, 0, 0, 0, 0, 0, 0 ...]
all; [3, 2, 0, 0, 0, 0, 0, 0, 0 ...]
ask: [0, 0, 0, 0, 0, 0, 0, 0, 0 ...]
dog: [0, 0, 0, 0, 0, 0, 0, 0, 2 ...]
...
```